

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-110 – SOFTWARE ENGINEERING – SPRING 2026

Final Project Report

Topic 4 - Async Research Assistant

Team: Paputu **Date:** May 23, 2026
Repo: <https://github.com/shahin1717/researchassistant>
Members: **Shahin Alakparov** (@shahin1717),
Raul Aghayev (@Raghavev17889),
Nicat Alaskarli (@NicatAlaskarli)

Executive Summary

We built an async research assistant that answers natural-language questions by querying Wikipedia, arXiv, and a configurable web-search provider in parallel, caching all source responses in SQLite, and synthesising a cited answer with a Gemini LLM. The headline concurrency result is a **4.0× speedup** (17.35s sequential → 4.37s parallel) measured across five research questions, confirming that network I/O is the dominant bottleneck and that `asyncio.gather` is the right tool. Our most important robustness story is **graceful degradation**: every source is wrapped in an individual `asyncio.timeout` and a Tenacity retry loop, so a single hanging or failing provider never blocks the answer-the pipeline synthesises from whichever sources succeeded. The most surprising lesson was that per-source timings (not just the total parallel time) are necessary for diagnosing bottlenecks: in typical runs arXiv and Tavily are 3× slower than Wikipedia, dominating the wall clock despite being parallel. If we started over we would add a **FailoverLLM** provider from day one rather than discovering the Gemini free-tier quota limit on submission day.

Contents

Executive Summary	1
1 Project Overview	3
1.1 Problem framing & scope	3
1.2 Provider choice and the AI module contract	3
2 Architecture	3
2.1 Module map	3
2.2 OOP design & key abstractions	4
2.3 Data flow across module boundaries	5
3 Concurrency & Performance	5
3.1 Why asyncio	5
3.2 Sequential-vs-concurrent benchmark	6
3.3 Rate limit & politeness	6
4 Robustness & Error Handling	6
4.1 Wrapping the AI module	6
4.2 Failure-mode analysis: Gemini 429	7
4.3 Input validation	7
5 Storage & Caching	7
5.1 Schema	7
5.2 Caching policy	8
6 Testing	8
6.1 Strategy & coverage	8
6.2 Notable tests	9
7 Deployment & Reproducibility	9
7.1 Docker image	9
7.2 Configuration	9
8 Limitations & Future Work	10
9 Tools & Acknowledgements	10
References	10
A Appendix - Reproducing the benchmark	11

1. Project Overview

1.1 Problem framing & scope

The user submits a research question via CLI or a Streamlit web interface. The system fetches relevant excerpts from three independent sources (Wikipedia, arXiv, and a web-search engine) and asks a large language model to produce a 3–6 sentence answer with inline numbered citations ([1], [2], etc.). The answer and references are printed to stdout; cost telemetry is stored in SQLite.

Inputs: a natural-language question string (1–500 characters); optional `-sources` flag restricting to a subset of providers; optional `-no-cache` flag.

Outputs: a formatted answer block with numbered references, logged per-source fetch timings, and a persistent cost entry in the telemetry database.

Entry points: `python -m src ask "..."` (CLI); `python -m src cost-report` (spend summary); `streamlit run src/app.py` (web UI).

We tightened the spec in two places: we used SQLite instead of PostgreSQL for the cache (Topic 4's TOPIC.md explicitly lists this as acceptable, and it removes infrastructure complexity from a single-container Docker image), and as HTTP API support is not required for Topic 4, we focused engineering effort on CLI, Streamlit UI, and core robustness instead. We did not implement the `-since 24h` time filter on `cost-report`, which is tracked as future work.

1.2 Provider choice and the AI module contract

LLM: Google Gemini (`gemini-2.0-flash` / `gemini-3.5-flash`), selected via `LLM_PROVIDER=gemini` and `LLM_MODEL` in `.env`. The module also ships Anthropic and OpenAI adapters; swapping providers requires changing two environment variables and no source edits.

Web search: Tavily (`WEB_SEARCH_PROVIDER=tavily`). DuckDuckGo is used as a fallback when no key is configured.

We call four functions from the provided `ai/` package: `ai.fetch_wikipedia`, `ai.fetch_arxiv`, `ai.fetch_web`, and `ai.synthesize`. **We did not modify the public interface of the provided `ai/` package.**

When the provider returns a valid `AnswerWithCitations` object, the synthesizer already strips hallucinated citation indices; our SE layer additionally checks `answer.citations` is a list before iterating, and surfaces a clean `RuntimeError` (not a stack trace) when `fetch_sources` is empty.

2. Architecture

2.1 Module map

```
CLI / Streamlit UI (src/cli.py, src/app.py)
    |
    v
src/core/researcher.py      <- business logic
    |
    |      v
    |      src/concurrency/orchestrator.py  <- asyncio.gather
    |      |
    |      [ai/ boundary]
    |      ai.fetch_wikipedia
```

```

    |   ai.fetch_arxiv
    |   ai.fetch_web
    |       |
    v       v
src/services/cache.py + ai_service.py  <- retries, TTL, rate limiter
    |
    v
src/storage/cache_store.py             <- SQLite (cache_entries, spend_log,
                                         cache_events)

```

The `ai/` package boundary is crossed only from `orchestrator.py` (for fetches) and `ai_service.py` (for synthesis). All other modules interact exclusively with SE-layer types (`Source`, `AnswerWithCitations` from `ai.schemas` are treated as read-only DTOs). Swapping providers (Anthropic → OpenAI) requires changing `LLM_PROVIDER` and `LLM_MODEL` in `.env`; zero source files change.

Module summary:

- `src/config.py` - Pydantic `Settings` reads all env vars and exposes typed fields; also initialises Python logging and forwards keys to the `ai/` package via `os.environ`.
- `src/models.py` - Frozen Pydantic models `ResearchSession` and `QueryResult` used by the SE layer; no naked dicts cross module boundaries.
- `src/concurrency/orchestrator.py` - `fetch_all()` wraps each source coroutine in `_timed()` with an `asyncio.timeout`, then fans out via `asyncio.gather`; records per-source wall-clock timings.
- `src/core/researcher.py` - Checks SQLite cache per source, calls `fetch_all` for misses, persists new results, calls `AIService.synthesize`, records cost telemetry.
- `src/services/ai_service.py` - `AIService` wraps every `ai.*` call with Tenacity exponential backoff, `asyncio.Semaphore`, per-call `asyncio.wait_for`, and an optional sliding-window `TokenBudgetLimiter`.
- `src/services/cache.py` - `QueryCache` normalises keys (`.strip().lower()`), delegates to `CacheStore`, and logs hit/miss events.
- `src/storage/cache_store.py` - Thread-safe SQLite with three tables: `cache_entries`, `spend_log`, `cache_events`. Exposes telemetry helpers (`spend_breakdown`, `most_expensive_queries`).
- `src/cli.py` - argparse CLI with `ask` and `cost-report` subcommands; validates question length and source aliases before any network call.
- `src/app.py` - Streamlit UI with a Research tab and a Developer Dashboard (cache stats, spend breakdown, expensive queries).

2.2 OOP design & key abstractions

The most important OOP decision in our SE layer is `TokenBudgetLimiter`, which uses **composition** inside `AIService`: the service holds a `TokenBudgetLimiter | None` instance and delegates the sliding-window bookkeeping to it.

```

1 class TokenBudgetLimiter:
2     """Sliding-window token budget (TPM) limiter."""
3
4     def __init__(self, tokens_per_window: int, *,
5                  window_seconds: float = 60.0,
6                  sleeper: Callable[[float], Awaitable[None]] = asyncio.sleep,
7                  clock: Callable[[], float] = time.monotonic) -> None:

```

```

8         self._tokens_per_window = tokens_per_window
9         self._window_seconds = window_seconds
10        self._sleep = sleeper
11        self._clock = clock
12        self._lock = asyncio.Lock()
13        self._events: deque[_TokenUsage] = deque()
14
15        async def acquire(self, tokens: int) -> float:
16            """Block until tokens are available. Returns seconds waited."""
17            ...
18
19    class AIService:
20        def __init__(self, *, token_budget_tpm: int | None = None, ...) -> None:
21            self._rate_limiter = (
22                None if token_budget_tpm <= 0
23                else TokenBudgetLimiter(token_budget_tpm, ...)
24            )

```

We chose composition over inheritance because `TokenBudgetLimiter` has no natural “is-a” relationship with `AIService` - it is a self-contained policy object. Making it injectable (via constructor parameters) also enables the test suite to pass a fake `sleeper` and `clock` without patching globals, which made the token-budget tests deterministic.

The provided `ai/` package already demonstrates the ABC/factory pattern (`LLMProvider` → `AnthropicProvider`, `OpenAIProvider`, `GeminiProvider`). We reference rather than replicate that pattern; our own example of a polymorphic design is `QueryCache`, which accepts any `CacheStore` instance, making it straightforward to substitute an in-memory store in tests.

2.3 Data flow across module boundaries

Our SE-layer dataclasses:

- `ResearchSession` - frozen Pydantic model carrying the validated question, source list, and cache config for a single request.
- `QueryResult` - frozen Pydantic model returned by `QueryCache.get/set`; carries the canonical key, raw JSON payload, and TTL metadata.

No naked dictionaries cross module boundaries. The `Source` and `AnswerWithCitations` types from `ai.schemas` are used as read-only DTOs within the SE layer without wrapping, because they are already well-typed frozen Pydantic models. We wrapped the cache payload (`QueryResult`) because the SE layer needed to add `cached`, `expires_at`, and `canonical_query` fields that the `ai/` schema does not provide.

3. Concurrency & Performance

3.1 Why asyncio

We used `asyncio` + `httpx` for all three source fetchers. The bottleneck is network I/O: each fetch is a single HTTP round-trip to an external API (Wikipedia REST, arXiv Atom, Tavily). Threads would work but carry heavier overhead and do not compose naturally with `asyncio.timeout`. `ProcessPoolExecutor` would be wrong because there is no CPU-bound computation in the fetch path.

Parallelism is bounded by `asyncio.Semaphore(MAX_PARALLEL)` (default 10) inside `AIService._run_async_call`. We chose 10 because each query generates at most three concurrent source calls, so the semaphore is effectively never the limiting factor for a single question; it protects against accidental misuse if the CLI is driven with a loop. With `Semaphore(1)` the system degrades to sequential; with

`Semaphore(100)` on a high-QPS workload it would exhaust provider rate limits and trigger retries.

A single shared `httpx.AsyncClient` is passed to all three fetchers per request, enabling HTTP connection reuse across sources.

3.2 Sequential-vs-concurrent benchmark

Workload	N	Sequential (s)	Concurrent (s)	Speedup
5 research questions (wiki + arxiv + web)	5	17.35	4.37	4.0×

Machine: MacBook Pro M1, Python 3.11, caches cleared, `scripts/bench.py`.

The theoretical maximum speedup for three equal-latency sources is 3×; we exceed this because Wikipedia is significantly faster than arXiv and Tavily. In the parallel case the wall clock equals the slowest source (~1.5s for Tavily), while the sequential case sums all three (wiki 0.5 + arxiv 1.4 + web 1.5 = 3.4s, multiplied across 5 questions). The new bottleneck is the LLM synthesis call (~5s per question), which is serial and dominates total end-to-end latency. The per-source timing breakdown is now visible in the logs:

```
INFO source_fetched source=wiki elapsed_s=0.48 count=3
INFO source_fetched source=arxiv elapsed_s=1.37 count=3
INFO source_fetched source=web elapsed_s=1.54 count=3
INFO fetch_all_complete timings={...total_parallel: 1.54} total_sources=9
```

3.3 Rate limit & politeness

Every `ai.*` call is wrapped in a Tenacity retry with exponential backoff (`wait_exponential(multiplier=0.5, min=0.5, max=5.0)`), 3 attempts). On HTTP 429, the `ProviderError` is caught and retried with back-off.

For token-per-minute enforcement, `TokenBudgetLimiter` maintains a sliding deque of (`timestamp`, `tokens`) events. Before each AI call it estimates the token count from argument character lengths (1 token ≈ 4 characters), prunes events older than 60s, and sleeps until headroom opens if the budget would be exceeded. The TPM limit is configurable via `TOKEN_BUDGET_TPM`; it defaults to 0 (disabled) so the free-tier demo does not stall. We observed a 429 from Gemini during development when the free-tier daily quota was exhausted; the logs showed three `ai_call_retry` warnings followed by `ai_call_failed`, and the CLI printed a clean `Error: Gemini call failed: 429 RESOURCE_EXHAUSTED` message.

4. Robustness & Error Handling

4.1 Wrapping the AI module

Every call to `ai.*` goes through `AIService._run_async_call`, which:

1. Acquires the token budget (sleeps if TPM would be exceeded).
2. Logs `ai_call_start` with `operation` and `timeout_seconds`.
3. Acquires `asyncio.Semaphore` to bound concurrency.
4. Calls `asyncio.wait_for(func(...), timeout=timeout_seconds)`.
5. On success, logs `ai_call_success` with `duration_ms`.

6. On `ProviderError` or `TimeoutError`, Tenacity retries up to 3 times with exponential back-off, logging `ai_call_retry` each time.
7. After all retries exhausted, logs `ai_call_failed` and re-raises.

Each source additionally sits inside a per-task `asyncio.timeout(per_source_timeout_seconds)` in `orchestrator.py`, so a slow Wikipedia response cannot hold arXiv and Tavily hostage regardless of how many retries are attempted. The synthesis call uses a separate 30s timeout.

Input validation before calling the AI module: question is stripped and checked non-empty and ≤ 500 characters; `sources` must contain at least one recognised alias (`wiki`, `arxiv`, `web`); empty `fetches_sources` raises `RuntimeError` before the synthesis call is even attempted.

4.2 Failure-mode analysis: Gemini 429

How we injected it: ran repeated queries until the Gemini free-tier daily quota was exhausted.

What the system did: Tenacity fired three attempts at 0.5s, 1s, and 2s back-off. All three returned 429. After the third attempt the exception propagated up through `researcher.py` to the CLI error handler.

What the user saw:

```
Error: Gemini call failed: 429 RESOURCE_EXHAUSTED. ...
* Quota exceeded for metric: .../generate_content_free_tier_requests
Please retry in 22s.
```

What the logs showed:

```
INFO ai_call_start operation=synthesize timeout_seconds=30.0
WARN ai_call_retry attempt=1 error=ProviderError
WARN ai_call_retry attempt=2 error=ProviderError
ERROR ai_call_failed operation=synthesize duration_ms=6482.3
ERROR research_failed error=Gemini call failed: 429 ...
```

The log was sufficient to diagnose the root cause without re-running the code. The fix is to set `LLM_PROVIDER=anthropic` in `.env` and supply an Anthropic key.

4.3 Input validation

- **CLI:** `validate_question()` rejects empty strings (“*Question cannot be empty.*”) and strings > 500 characters (“*Question must be 500 characters or fewer.*”). `parse_sources()` rejects unknown source aliases with a list of allowed values.
- **Env:** Pydantic `Settings` with typed fields; missing required keys produce a `ValidationError` on startup with a clear field name.
- **Cache layer:** `QueryCache._normalize_source()` raises `ValueError` on empty source strings. `CacheStore.record_cache_event()` raises `ValueError` if event is not ‘hit’ or ‘miss’.
- **Models:** `ResearchSession` and `QueryResult` use `ConfigDict(extra="forbid")` and field validators to reject blank strings.

5. Storage & Caching

5.1 Schema

```

CREATE TABLE IF NOT EXISTS cache_entries (
    source          TEXT NOT NULL,
    canonical_query TEXT NOT NULL,
    response_json   TEXT NOT NULL,    -- JSON array of ai.schemas.Source
    dicts
    created_at      REAL NOT NULL,    -- Unix timestamp (time.time())
    PRIMARY KEY (source, canonical_query)
);

CREATE TABLE IF NOT EXISTS spend_log (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    source          TEXT NOT NULL,    -- provider name (e.g. "synthesis")
    canonical_query TEXT NOT NULL,
    cost_usd        REAL NOT NULL,
    created_at      REAL NOT NULL
);

CREATE TABLE IF NOT EXISTS cache_events (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    source          TEXT NOT NULL,
    canonical_query TEXT NOT NULL,
    event           TEXT NOT NULL CHECK(event IN ('hit', 'miss')),
    created_at      REAL NOT NULL
);

```

`response_json` stores a JSON-serialised `list[Source.model_dump()]`. Switching embedding providers or source schemas requires a cache flush, not a migration. We chose text/JSON over binary blobs because the payloads are small and human-readable, which eases debugging. `CacheStore` is thread-safe via `threading.RLock`.

5.2 Caching policy

We cache per `(source, canonical_query)` pair with a configurable TTL (default 24h). The canonical query is `query.strip().lower()`, so "WHAT IS PHOTOSYNTHESIS?" and "what is photosynthesis" hit the same row. `cleanup_expired()` is called on every `cache.get()` to remove stale rows before the lookup.

In a typical demo run (5 questions, first time), all 15 source lookups are misses. On a second run without `-no-cache` all 15 are hits, reducing total latency from ≈ 35 s to ≈ 25 s (only synthesis calls remain). The hit rate from `artefacts/demo_output.txt` (second run): $15/15 = 100\%$.

The main staleness risk is that web-search results change over time. A news-related question cached yesterday may omit today's events. Production hardening would require shorter TTLs for web sources and a freshness indicator in the answer.

6. Testing

6.1 Strategy & coverage

Suite	Coverage
src/ (SE layer, "pytest -cov src/")	86%
Provided ai/ smoke tests	passing

All 51 tests run offline. The `ai.*` functions are mocked via `unittest.mock.AsyncMock` and custom `FakeAIService` classes in `conftest.py`. HTTP is not directly mocked because all HTTP passes through the provided `ai/` package, which is itself mocked at the `ai.fetch_*` boundary. The three

main uncovered areas are `src/app.py` (Streamlit UI, not unit-testable), `src/cli_demo.py` (demo script), and some late error-path branches in `ai_service.py`.

Provided smoke tests (`tests/test_ai_smoke.py`) pass unchanged.

6.2 Notable tests

(i) **Happy-path end-to-end** (`test_se_layer.py::test_researcher_returns_cited_answer`): mocks all three sources returning one `Source` each and verifies the final `AnswerWithCitations` has a non-empty answer and at least one citation. Exercises the full stack from `researcher.research()` down to the mock synthesizer.

(ii) **Concurrency / graceful degradation** (`test_se_layer.py::test_orchestrator_per_task_timeout_isolates`): patches `arXiv` to `asyncio.sleep(9999)` and sets `per_source_timeout_seconds = 0.05`. Asserts that Wikipedia results still appear in the output and that the total wall-clock is under 1s. This test proved the per-task `asyncio.timeout` was necessary-before adding it, the test hung for the full 9999 seconds.

(iii) **Error-path coverage** (`test_ai_service.py::test_token_budget_limiter_waits_when_budget_exhausted`): fills the token budget to capacity, then calls `acquire(1)` and asserts the limiter slept at least once. Caught a bug where `_seconds_until_available` returned a negative value for an already-expired window, causing an infinite loop.

7. Deployment & Reproducibility

7.1 Docker image

```
# Build
docker build -t research-assistant .

# Run CLI
docker run --env-file .env research-assistant \
  python -m src ask "What is photosynthesis?"

# Run Streamlit UI
docker run --env-file .env -p 8501:8501 research-assistant \
  streamlit run src/app.py --server.address 0.0.0.0
```

The Dockerfile uses a **two-stage build**: a `python:3.11-slim` builder installs dependencies into `/opt/venv`; the runtime stage copies only the venv and application code. The image runs as a non-root appuser. Python version: 3.11-slim.

7.2 Configuration

Variable	Required	Missing behaviour
LLM_PROVIDER	no (default: <code>gemini</code>)	Uses Gemini
LLM_MODEL	no (default: <code>gemini-2.0-flash</code>)	Uses default model
GOOGLE_API_KEY	if Gemini	API calls fail with 401
ANTHROPIC_API_KEY	if Anthropic	API calls fail with 401
OPENAI_API_KEY	if OpenAI	API calls fail with 401
WEB_SEARCH_PROVIDER	no (default: <code>tavily</code>)	Falls back to DuckDuckGo
TAVILY_API_KEY	if Tavily	Web search disabled or DDG used
CACHE_DIR	no (default: <code>./.cache</code>)	Creates directory automatically
CACHE_TTL_SECONDS	no (default: 86400)	24-hour TTL
PER_SOURCE_TIMEOUT_SECONDS	no (default: 10)	10-second per-source cap
MAX_PARALLEL	no (default: 10)	Semaphore bound of 10
LOG_LEVEL	no (default: <code>INFO</code>)	INFO logging
TOKEN_BUDGET_TPM	no (default: 0)	Rate limiter disabled

8. Limitations & Future Work

- **Free-tier LLM quota.** The system uses Gemini’s free tier by default, which has a per-day request cap. On submission day we hit this limit. Production deployment requires a paid API key or a configured fallback provider. A `FailoverLLM` wrapper (bonus +3) would have resolved this automatically.
- **Wikipedia often returns zero results.** The Wikipedia REST API regularly returns an empty list for technical queries. We cache the empty result, which is correct but means subsequent runs also show 0 Wikipedia sources. A smarter retry with a shorter query would help.
- **No -since filter on cost-report.** The spend log stores timestamps but the CLI does not yet expose time-range filtering. All spend since the database was created is reported.
- **SQLite in a multi-process deployment.** The current `RLock` strategy is correct for a single-process async application. A multi-worker deployment (e.g., Gunicorn + Uvicorn workers) would require migrating to PostgreSQL or a shared Redis cache.

9. Tools & Acknowledgements

AI-assisted contributions:

- `src/concurrency/orchestrator.py` - Claude (Anthropic) suggested the `_timed()` coroutine wrapper pattern and the placement of `asyncio.timeout()` per task. The team reviewed and understands every line; specifically, that `_timed` catches all exceptions so `asyncio.gather` needs no `return_exceptions=True`.
- `tests/test_storage.py` and `tests/test_researcher.py` - Claude scaffolded the initial test structure; the team added domain-specific assertions and corrected the `confest.py` fixture references.
- `scripts/demo.py` - Claude drafted the script; the team integrated it with the actual `format_answer` helper and verified it against the live demo.
- `src/config.py` (`_ExtraFormatter`) - Claude proposed the custom log formatter that appends `extra={}` fields to log lines. The team tested and deployed it.

All other modules (`ai_service.py`, `cache_store.py`, `researcher.py`, `cli.py`, `app.py`) were written by the team with standard IDE autocompletion assistance.

Libraries: `tenacity` (retries), `pydantic-settings` (config), `httpx` (async HTTP), `pytest-asyncio` (async tests), `streamlit` (web UI).

References

- [1] Anthropic API documentation. <https://docs.anthropic.com>
- [2] `tenacity` retry library. <https://tenacity.readthedocs.io>
- [3] `pydantic-settings` documentation. https://docs.pydantic.dev/latest/concepts/pydantic_settings/
- [4] Python `asyncio` documentation: `asyncio.timeout`, `asyncio.gather`. <https://docs.python.org/3/library/asyncio.html>

[5] Tavily AI Search API. <https://docs.tavily.com>

[6] Google Gemini API. <https://ai.google.dev/gemini-api/docs>

A. Appendix - Reproducing the benchmark

```
git clone https://github.com/shahin1717/researchassistant
cd researchassistant
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # add GOOGLE_API_KEY and TAVILY_API_KEY
python scripts/bench.py
```

The benchmark runs 5 questions sequentially (one source at a time) and then in parallel (`asyncio.gather`), printing wall-clock seconds and speedup per question. Numbers in the report were measured on a MacBook Pro M1 with a stable broadband connection. Provider latency varies; expect $\pm 20\%$ variance.

End of report - thank you for reading.